

Index

Classes

◦ KiteConnect

- __init__
- basket_order
- cancel_mf_order
- cancel_mf_subscription
- cancel_order
- convert_pos
- delete_gtt
- exit_order
- generate_session_key
- get_gtt
- get_gtts
- historical_data
- holdings
- instruments
- invalidate_session
- invalidate_session_key
- login_url
- ltp
- margins
- mf_holdings
- mf_instruments
- mf_orders
- mf_sips

kiteconnect module

Kite Connect API client for Python -
- <https://kite.trade> (kite.trade).

Zerodha Technology Pvt. Ltd. (c)
2021

License

KiteConnect Python library is
licensed under the MIT License

The library

Kite Connect is a set of REST-like APIs that expose many capabilities required to build a complete investment and trading platform. Execute orders in real time, manage user portfolio, stream live market data (WebSockets), and more, with the simple HTTP API collection

- modify_gtt
- modify_mf_s
- modify_order
- ohlcv
- order_history
- order_margin
- order_trades
- orders
- place_gtt
- place_mf_order
- place_mf_sell
- place_order
- positions
- profile
- quote
- renew_access
- set_access
- set_session
- trades
- trigger_rar

◦ **KiteTicker**

- __init__
- close
- connect
- is_connected
- resubscribe
- set_mode
- stop
- stop_retry

This module provides an easy to use abstraction over the HTTP APIs. The HTTP calls have been converted to methods and their JSON responses are returned as native Python structures, for example, dicts, lists, bools etc. See the **Kite Connect API documentation** (<https://kite.trade/docs/connect/v3/>) for the complete list of APIs, supported parameters and values, and response formats.

- subscribe
- unsubscribe

Sub-modules

- kiteconnect.exc
(exceptions.m.h

Getting started

```
#!/python
import logging
from
kiteconnect
import
KiteConnect

logging.basicConfig
onfig(level=lo
gging.DEBUG)

kite =
KiteConnect(ap
i_key="your_ap
i_key")

# Redirect the
user to the
login url
obtained
# from
kite.login_url
(), and
receive the
request_token
# from the
registered
```

```
redirect url  
after the  
login flow.  
# Once you  
have the  
request_token,  
obtain the  
access_token  
# as follows.
```

```
data =  
kite.generate_  
session("reque  
st_token_here"  
,  
api_secret="yo  
ur_secret")  
kite.set_acces  
s_token(data["  
access_token"]  
)
```

```
# Place an  
order  
try:  
    order_id =  
kite.place_ord  
er(variety=kit  
e.VARIETY_REGU
```

LAR,

tradingsymbol=
"INFY",

exchange=kite.
EXCHANGE_NSE,

transaction_ty
pe=kite.TRANSA
CTION_TYPE_BUY
,

quantity=1,

order_type=kit
e.ORDER_TYPE_M
ARKET,

product=kite.P
RODUCT_CNC,

validity=kite.
VALIDITY_DAY)

logging.info("
Order placed.
ID is:

```
{}".format(ord  
er_id))  
except  
Exception as  
e:
```

```
logging.info("  
Order  
placement  
failed:  
{}".format(e.m  
essage))
```

```
# Fetch all  
orders  
kite.orders()
```

```
# Get  
instruments  
kite.instrumen  
ts()
```

```
# Place an  
mutual fund  
order  
kite.place_mf_  
order(  
  

```

```
tradingsymbol=
```

```
"INF090I01239"
```

```
,
```

```
transaction_type=kite.TRANSACTION_TYPE_BUY
```

```
,
```

```
amount=5000,
```

```
tag="mytag"
```

```
)
```

```
# Cancel a mutual fund order  
kite.cancel_mf_order(order_id="order_id")
```

```
# Get mutual fund instruments  
kite.mf_instruments()
```

A typical web application

In a typical web application where a new instance of views, controllers etc. are created per incoming HTTP request, you will need to initialise a new instance of Kite client per request as well. This is because each individual instance represents a single user that's authenticated, unlike an **admin** API where you may use one instance to manage many users.

Hence, in your web application, typically:

- You will initialise an instance of the Kite client
- Redirect the user to the `login_url()`
- At the redirect url endpoint, obtain the `request_token` from the query parameters
- Initialise a new instance of Kite client, use `generate_session()` to obtain the `access_token`

along with authenticated user data

- Store this response in a session and use the stored `access_token` and initialise instances of Kite client for subsequent API calls.

Exceptions

Kite Connect client saves you the hassle of detecting API errors by looking at HTTP codes or JSON error responses. Instead, it raises aptly named **exceptions** (**exceptions.m.html**) that you can catch.

[SHOW SOURCE](#) ≡

Classes

```
class KiteConnect
```

The Kite Connect API wrapper class.

In production, you may initialise a single instance of this class per `api_key`.

Ancestors (in MRO)

- KiteConnect
- __builtin__.object

Class variables

var EXCHANGE_BFO

var EXCHANGE_BSE

var EXCHANGE_CDS

var EXCHANGE_MCX

var EXCHANGE_NFO

var EXCHANGE_NSE

var GTT_STATUS_ACTIVE

**var
GTT_STATUS_CANCELLED**

var GTT_STATUS_DELETED

var GTT_STATUS_DISABLED

var GTT_STATUS_EXPIRED

var GTT_STATUS_REJECTED

**var
GTT_STATUS_TRIGGERED**

var GTT_TYPE_OCO

var GTT_TYPE_SINGLE

var MARGIN_COMMODITY

var MARGIN_EQUITY

var ORDER_TYPE_LIMIT

var ORDER_TYPE_MARKET

var ORDER_TYPE_SL

var ORDER_TYPE_SLM

var POSITION_TYPE_DAY

**var
POSITION_TYPE_OVERNIGHT**

var PRODUCT_BO

var PRODUCT_CNC

var PRODUCT_CO

var PRODUCT_MIS

var PRODUCT_NRML

var STATUS_CANCELLED

var STATUS_COMPLETE

var STATUS_REJECTED

**var
TRANSACTION_TYPE_BUY**

**var
TRANSACTION_TYPE_SELL**

var VALIDITY_DAY

var VALIDITY_IOC

var VARIETY_AMO

var VARIETY_BO

var VARIETY_CO

```
var VARIETY_REGULAR
```

Instance variables

```
var access_token
```

```
var api_key
```

```
var debug
```

```
var disable_ssl
```

```
var proxies
```

```
var root
```

```
var session_expiry_hook
```

```
var timeout
```

Methods

```
def __init__(self, api_key,
               access_token=None,
               root=None,
               debug=False,
               timeout=None,
               proxies=None,
               pool=None,
               disable_ssl=False)
```

Initialise a new Kite Connect client instance.

- `api_key` is the key issued to you
- `access_token` is the token obtained after the login flow in exchange for the `request_token`. Pre-login, this will default to `None`, but once you have obtained it, you should persist it in a database or session to pass to the Kite Connect class initialisation for subsequent requests.

- root is the API endpoint root. Unless you explicitly want to send API requests to a non-default endpoint, this can be ignored.
- debug, if set to True, will serialise and print requests and responses to stdout.
- timeout is the time (seconds) for which the API client will wait for a request to complete before it fails. Defaults to 7 seconds
- proxies to set requests proxy. Check [python requests documentation](http://docs.python-requests.org/en/master/user/advanced/#proxies) (<http://docs.python-requests.org/en/master/user/advanced/#proxies>) for usage and examples.
- pool is manages request pools. It takes a dict of params accepted by HTTPAdapter as

described here in

[python requests](#)

[documentation](#)

[.http://docs.python-](http://docs.python-requests.org/en/master/api/#requests.adapters.HTTPAdapter)

[requests.org/en/master/api/#requests.adapters.HTTPAdapter](http://docs.python-requests.org/en/master/api/#requests.adapters.HTTPAdapter))

- `disable_ssl` disables the SSL verification while making a request. If set requests won't throw `SSLError` if its set to custom root url without SSL.

[SHOW SOURCE](#) ≡

```
def basket_order_margins(self, params,  
                           consider_positions=True,  
                           mode=None)
```

Calculate total margins
required for basket of orders
including margin benefits

- `params` is list of orders to fetch basket margin
- `consider_positions` is a boolean to consider users positions

- mode is margin
response mode type.
compact - Compact
mode will only give the
total margins

SHOW SOURCE ≡

```
def cancel_mf_order(self,  
                    order_id)
```

Cancel a mutual fund order.

SHOW SOURCE ≡

```
def cancel_mf_sip(self,  
                  sip_id)
```

Cancel a mutual fund SIP.

SHOW SOURCE ≡

```
def cancel_order(self, variety,  
                 order_id,  
                 parent_order_id=None)
```

Cancel an order.

SHOW SOURCE ≡

```
def convert_position(self, exchange,
                      tradingsymbol,
                      transaction_type,
                      position_type,
                      quantity,
                      old_product,
                      new_product)
```

Modify an open position's
product type.

[SHOW SOURCE](#) ≡

```
def delete_gtt(self,
                trigger_id)
```

Delete a GTT order.

[SHOW SOURCE](#) ≡

```
def exit_order(self, variety,
                order_id,
                parent_order_id=None)
```

Exit a BO/CO order.

[SHOW SOURCE](#) ≡

```
def generate_session(self,
                     request_token,
                     api_secret)
```

Generate user session details like `access_token` etc by exchanging `request_token`. Access token is automatically set if the session is retrieved successfully.

Do the token exchange with the `request_token` obtained after the login flow, and retrieve the `access_token` required for all subsequent requests. The response contains not just the `access_token`, but metadata for the user who has authenticated.

- `request_token` is the token obtained from the GET paramers after a successful login redirect.
- `api_secret` is the API `api_secret` issued with the API key.

[SHOW SOURCE](#) ≡

```
def get_gtt(self,  
            trigger_id)
```

Fetch details of a GTT

SHOW SOURCE ≡

```
def get_gtts(self)
```

Fetch list of gtt existing in an account

SHOW SOURCE ≡

```
def historical_data(self,  
                    instrument_token,  
                    from_date,  
                    to_date,  
                    interval,  
                    continuous=False,  
                    oi=False)
```

Retrieve historical data
(candles) for an instrument.

Although the actual response
JSON from the API does not
have field names such as
'open', 'high' etc., this
function call structures the

data into an array of objects with field names. For example:

- `instrument_token` is the instrument identifier (retrieved from the `instruments()` call).
- `from_date` is the From date (datetime object or string in format of `yyyy-mm-dd HH:MM:SS`).
- `to_date` is the To date (datetime object or string in format of `yyyy-mm-dd HH:MM:SS`).
- `interval` is the candle interval (minute, day, 5 minute etc.).
- `continuous` is a boolean flag to get continuous data for futures and options instruments.
- `oi` is a boolean flag to get open interest.

[SHOW SOURCE](#) ≡

```
def holdings(self)
```

Retrieve the list of equity holdings.

[SHOW SOURCE](#) ≡

```
def instruments(self,  
                exchange=None)
```

Retrieve the list of market instruments available to trade.

Note that the results could be large, several hundred KBs in size, with tens of thousands of entries in the list.

- exchange is specific exchange to fetch (Optional)

[SHOW SOURCE](#) ≡

```
def invalidate_access_token(self,  
                            access_token=None)
```

Kill the session by invalidating the access token.

- `access_token` to invalidate. Default is the active `access_token`.

SHOW SOURCE ≡

```
def invalidate_refresh_token(self,  
                             refresh_token)
```

Invalidate refresh token.

- `refresh_token` is the token which is used to renew access token.

SHOW SOURCE ≡

```
def login_url(self)
```

Get the remote login url to which a user should be redirected to initiate the login flow.

SHOW SOURCE ≡

```
def ltp(self,  
        *instruments)
```

Retrieve last price for list of instruments.

- instruments is a list of instruments, Instrument are in the format of exchange:trading symbol. For example NSE:INFY

SHOW SOURCE ≡

```
def margins(self,  
            segment=None)
```

Get account balance and cash margin details for a particular segment.

- segment is the trading segment (eg: equity or commodity)

SHOW SOURCE ≡

```
def mf_holdings(self)
```

Get list of mutual fund holdings.

SHOW SOURCE ≡

```
def mf_instruments(self)
```

Get list of mutual fund instruments.

SHOW SOURCE ≡

```
def mf_orders(self,  
               order_id=None)
```

Get all mutual fund orders or individual order info.

SHOW SOURCE ≡

```
def mf_sips(self,  
             sip_id=None)
```

Get list of all mutual fund SIP's or individual SIP info.

SHOW SOURCE ≡

```
def modify_gtt(self,  
                trigger_id,  
                trigger_type,  
                tradingsymbol,  
                exchange,  
                trigger_values,  
                last_price,  
                orders)
```

Modify GTT order

- `trigger_type` The type of GTT order(single/two-leg).
- `tradingsymbol` Trading symbol of the instrument.
- `exchange` Name of the exchange.
- `trigger_values` Trigger values (json array).
- `last_price` Last price of the instrument at the time of order placement.
- `orders` JSON order array containing following fields
 - `transaction_type` BUY or SELL
 - `quantity` Quantity to transact
 - `price` The min or max price to execute the order at (for LIMIT orders)

SHOW SOURCE ≡

```
def modify_mf_sip(self, sip_id,  
                  amount=None,  
                  status=None,  
                  instalments=None,  
                  frequency=None,  
                  instalment_day=None)
```

Modify a mutual fund SIP.

SHOW SOURCE ≡

```
def modify_order(self, variety, order_id,  
                 parent_order_id=None,  
                 quantity=None,  
                 price=None,  
                 order_type=None,  
                 trigger_price=None,  
                 validity=None,  
                 disclosed_quantity=None)
```

Modify an open order.

SHOW SOURCE ≡

```
def ohlc(self,  
         *instruments)
```

Retrieve OHLC and market
depth for list of instruments.

- instruments is a list of instruments, Instrument are in the format of exchange:trading symbol. For example NSE:INFY

SHOW SOURCE ≡

```
def order_history(self,  
                    order_id)
```

Get history of individual order.

- order_id is the ID of the order to retrieve order history.

SHOW SOURCE ≡

```
def order_margins(self,  
                    params)
```

Calculate margins for requested order list considering the existing positions and open orders

- params is list of orders to retrieve margins detail

SHOW SOURCE ≡

```
def order_trades(self,  
                  order_id)
```

Retrieve the list of trades
executed for a particular
order.

- order_id is the ID of
the order to retrieve
trade history.

SHOW SOURCE ≡

```
def orders(self)
```

Get list of orders.

SHOW SOURCE ≡

```
def place_gtt(self,  
               trigger_type,  
               tradingsymbol,  
               exchange,  
               trigger_values,  
               last_price,  
               orders)
```

Place GTT order

- `trigger_type` The type of GTT order(single/two-leg).
- `tradingsymbol` Trading symbol of the instrument.
- `exchange` Name of the exchange.
- `trigger_values` Trigger values (json array).
- `last_price` Last price of the instrument at the time of order placement.
- `orders` JSON order array containing following fields
 - `transaction_type` BUY or SELL
 - `quantity` Quantity to transact
 - `price` The min or max price to execute the order at (for LIMIT orders)

[SHOW SOURCE](#) ≡

```
def place_mf_order(self,  
                    tradingsymbol,  
                    transaction_type,  
                    quantity=None,  
                    amount=None,  
                    tag=None)
```

Place a mutual fund order.

[SHOW SOURCE](#) ≡

```
def place_mf_sip(self, tradingsymbol,  
                  amount, instalments,  
                  frequency,  
                  initial_amount=None,  
                  instalment_day=None,  
                  tag=None)
```

Place a mutual fund SIP.

[SHOW SOURCE](#) ≡

```
def place_order(self, variety, exchange,
                tradingsymbol,
                transaction_type,
                quantity, product,
                order_type, price=None,
                validity=None,
                disclosed_quantity=None,
                trigger_price=None,
                squareoff=None,
                stoploss=None,
                trailing_stoploss=None,
                tag=None)
```

Place an order.

[SHOW SOURCE](#) ≡

```
def positions(self)
```

Retrieve the list of positions.

[SHOW SOURCE](#) ≡

```
def profile(self)
```

Get user profile details.

[SHOW SOURCE](#) ≡

```
def quote(self,
          *instruments)
```

Retrieve quote for list of instruments.

- instruments is a list of instruments, Instrument are in the format of exchange:tradingsymbol. For example NSE:INFY

SHOW SOURCE ≡

```
def renew_access_token(self,
                        refresh_token,
                        api_secret)
```

Renew expired refresh_token using valid refresh_token.

- refresh_token is the token obtained from previous successful login flow.
- api_secret is the API api_secret issued with the API key.

SHOW SOURCE ≡

```
def set_access_token(self,  
                      access_token)
```

Set the access_token
received after a successful
authentication.

[SHOW SOURCE](#) ≡

```
def set_session_expiry_hook(self,  
                             method)
```

Set a callback hook for
session (TokenError --
timeout, expiry etc.) errors.

An access_token (login
session) can become invalid
for a number of reasons, but
it doesn't make sense for the
client to try and catch it
during every API call.

A callback method that
handles session errors can
be set here and when the
client encounters a token
error at any point, it'll be
called.

This callback, for instance, can log the user out of the UI, clear session cookies, or initiate a fresh login.

[SHOW SOURCE](#) ≡

def trades(self)

Retrieve the list of trades executed (all or ones under a particular order).

An order can be executed in tranches based on market conditions. These trades are individually recorded under an order.

[SHOW SOURCE](#) ≡

**def trigger_range(self,
 transaction_type,
 *instruments)**

Retrieve the buy/sell trigger range for Cover Orders.

[SHOW SOURCE](#) ≡

class KiteTicker

The WebSocket client for
connecting to Kite Connect's
streaming quotes service.

Getting started:

```
#!/python
import
logging
from
kiteconnect
import
KiteTicker

logging.basicConfig
Config(level=
logging.DEBUG
)

# Initialise
kws =
KiteTicker("y
our_api_key",
"your_access_
token")

def
on_ticks(ws,
ticks):
    #
    Callback to
    receive
    ticks.
```

```
logging.debug  
("Ticks:  
{ }".format(ti  
cks))
```

```
def  
on_connect(ws  
, response):
```

```
    #
```

```
    Callback on  
    successful  
    connect.
```

```
    #
```

```
    Subscribe to  
    a list of  
    instrument_to  
    kens  
    (RELIANCE and  
    ACC here).
```

```
ws.subscribe(  
    [738561,  
    5633])
```

```
    # Set  
    RELIANCE to  
    tick in  
    `full` mode.
```

```
ws.set_mode(w  
s.MODE_FULL,  
[738561])
```

```
def  
on_close(ws,  
code,  
reason):  
    # On  
connection  
close stop  
the event  
loop.  
    #  
Reconnection  
will not  
happen after  
executing  
`ws.stop()`  
    ws.stop()
```

```
# Assign the  
callbacks.  
kws.on_ticks  
= on_ticks  
kws.on_conne  
ct =  
on_connect
```

```
kws.on_close
= on_close

# Infinite
loop on the
main thread.
Nothing after
this will
run.
# You have to
use the pre-
defined
callbacks to
manage
subscriptions
.
kws.connect()
```

Callbacks

In below examples ws is the currently initialised WebSocket object.

- `on_ticks(ws, ticks)` - Triggered when ticks are received.
 - `ticks` - List of tick object. Check below

for sample structure.

- `on_close(ws, code, reason)` - Triggered when connection is closed.
 - `code` - WebSocket standard close event code
(<https://developer.mozilla.org/en-US/docs/Web/API/CloseEvent>)
 - `reason` - DOMString indicating the reason the server closed the connection
- `on_error(ws, code, reason)` - Triggered when connection is closed with an error.
 - `code` - WebSocket standard close event code
(<https://developer.mozilla.org/en-US/docs/Web/API/CloseEvent>)
 - `reason` - DOMString indicating the reason the server closed the connection

- `on_connect` - Triggered when connection is established successfully.
 - `response` - Response received from server on successful connection.
- `on_message(ws, payload, is_binary)` - Triggered when message is received from the server.
 - `payload` - Raw response from the server (either text or binary).
 - `is_binary` - Bool to check if response is binary type.
- `on_reconnect(ws, attempts_count)` - Triggered when auto reconnection is attempted.
 - `attempts_count` - Current reconnect attempt number.

- `on_noreconnect(ws)` - Triggered when number of auto reconnection attempts exceeds `reconnect_tries`.
- `on_order_update(ws, data)` - Triggered when there is an order update for the connected user.

Tick structure (passed to the

on_ticks callback)

```
[{  
  
  'instrument_t  
oken':  
53490439,  
    'mode':  
    'full',  
    'volume':  
12510,  
  
  'last_price':  
4084.0,  
  
  'average_pric  
e': 4086.55,  
  
  'last_quantit  
y': 1,  
  
  'buy_quantity  
' : 2356  
  
  'sell_quantit  
y': 2440,  
    'change':  
0.46740467404  
674046,
```

```
'last_trade_time':
datetime.datetime(2018, 1,
15, 13, 16,
54),

'timestamp':
datetime.datetime(2018, 1,
15, 13, 16,
56),
    'oi':
21845,

'oi_day_low':
0,

'oi_day_high'
: 0,
    'ohlcv': {

'high':
4093.0,

'close':
4065.0,
```

```
'open':  
4088.0,  
  
'low': 4080.0  
    },  
  
'tradable':  
True,  
    'depth':  
{  
  
    'sell': [{  
  
    'price':  
4085.0,  
  
    'orders':  
1048576,  
  
    'quantity':  
43  
        }, {  
  
    'price':  
4086.0,  
  
    'orders':  
2752512,
```

```
'quantity':  
134  
    }, {
```

```
'price':  
4087.0,
```

```
'orders':  
1703936,
```

```
'quantity':  
133  
    }, {
```

```
'price':  
4088.0,
```

```
'orders':  
1376256,
```

```
'quantity':  
70  
    }, {
```

```
'price':  
4089.0,
```

```
'orders':  
1048576,
```

```
'quantity':  
46  
    }],  
  
'buy': [{  
  
    'price':  
4084.0,  
  
    'orders':  
589824,  
  
    'quantity':  
53  
        }, {  
  
    'price':  
4083.0,  
  
    'orders':  
1245184,  
  
    'quantity':  
145  
        }, {  
  
    'price':  
4082.0,
```

```
'orders':  
1114112,  
  
'quantity':  
63  
      }, {
```

```
'price':  
4081.0,
```

```
'orders':  
1835008,  
  
'quantity':  
69  
      }, {
```

```
'price':  
4080.0,
```

```
'orders':  
2752512,  
  
'quantity':  
89  
      }]  
}  
  
,
```

```
...,  
...]
```

Auto reconnection

Auto reconnection is enabled by default and it can be disabled by passing reconnect param while initialising KiteTicker. On a side note, reconnection mechanism cannot happen if event loop is terminated using stop method inside on_close callback.

Auto reconnection mechanism is based on Exponential backoff (https://en.wikipedia.org/wiki/Exponential_backoff) algorithm in which next retry interval will be increased exponentially. reconnect_max_delay and reconnect_max_tries params can be used to tweak the algorithm where reconnect_max_delay is the maximum delay after which subsequent reconnection interval will become constant and

`reconnect_max_tries` is maximum number of retries before its quitting reconnection.

For example if `reconnect_max_delay` is 60 seconds and `reconnect_max_tries` is 50 then the first reconnection interval starts from minimum interval which is 2 seconds and keep increasing up to 60 seconds after which it becomes constant and when reconnection attempt is reached upto 50 then it stops reconnecting.

method `stop_retry` can be used to stop ongoing reconnect attempts and `on_reconnect` callback will be called with current reconnect attempt and `on_noreconnect` is called when reconnection attempts reaches max retries.

[SHOW SOURCE](#) ≡

Ancestors (in MRO)

- [KiteTicker](#)

- `__builtin__.object`

Class variables

`var CONNECT_TIMEOUT`

`var EXCHANGE_MAP`

`var MODE_FULL`

`var MODE_LTP`

`var MODE_QUOTE`

`var RECONNECT_MAX_DELAY`

`var RECONNECT_MAX_TRIES`

`var ROOT_URI`

Instance variables

`var connect_timeout`

`var debug`

`var on_close`

`var on_connect`

`var on_error`

`var on_message`

`var on_noreconnect`

`var on_open`

`var on_order_update`

`var on_reconnect`

`var on_ticks`

`var root`

`var socket_url`

`var subscribed_tokens`

Methods

```
def __init__(self, api_key,  
              access_token,  
              debug=False, root=None,  
              reconnect=True,  
              reconnect_max_tries=50,  
              reconnect_max_delay=60,  
              connect_timeout=30)
```

Initialise websocket client instance.

- `api_key` is the API key issued to you
- `access_token` is the token obtained after the login flow in exchange for the `request_token`. Pre-login, this will default to `None`, but once you have obtained it, you should persist it in a database or session to pass to the Kite Connect class initialisation for subsequent requests.
- `root` is the websocket API end point root. Unless you explicitly want to send API requests to a non-default endpoint, this can be ignored.
- `reconnect` is a boolean to enable WebSocket

autoreconnect in case of network failure/disconnection.

- `reconnect_max_delay` in seconds is the maximum delay after which subsequent reconnection interval will become constant. Defaults to 60s and minimum acceptable value is 5s.
- `reconnect_max_tries` is maximum number reconnection attempts. Defaults to 50 attempts and maximum up to 300 attempts.
- `connect_timeout` in seconds is the maximum interval after which connection is considered as timeout. Defaults to 30s.

[SHOW SOURCE](#) ≡

```
def close(self,  
        code=None,  
        reason=None)
```

Close the WebSocket
connection.

[SHOW SOURCE](#) ≡

```
def connect(self, threaded=False,  
            disable_ssl_verification=False,  
            proxy=None)
```

Establish a websocket
connection.

- `threaded` is a boolean indicating if the websocket client has to be run in threaded mode or not
- `disable_ssl_verification` disables building ssl context
- `proxy` is a dictionary with keys `host` and `port` which denotes the proxy settings

[SHOW SOURCE](#) ≡

```
def is_connected(self)
```

Check if WebSocket
connection is established.

[SHOW SOURCE](#) ≡

```
def resubscribe(self)
```

Resubscribe to all current
subscribed tokens.

[SHOW SOURCE](#) ≡

```
def set_mode(self, mode,  
              instrument_tokens)
```

Set streaming mode for the
given list of tokens.

- mode is the mode to
set. It can be one of the
following class
constants: MODE_LTP,
MODE_QUOTE, or
MODE_FULL.
- instrument_tokens is
list of instrument
tokens on which the
mode should be
applied

[SHOW SOURCE](#) ≡

def stop(self)

Stop the event loop. Should be used if main thread has to be closed in on_close method. Reconnection mechanism cannot happen past this method

[SHOW SOURCE](#) ≡

def stop_retry(self)

Stop auto retry when it is in progress.

[SHOW SOURCE](#) ≡

**def subscribe(self,
 instrument_tokens)**

Subscribe to a list of instrument_tokens.

- instrument_tokens is list of instrument_tokens to subscribe

[SHOW SOURCE](#) ≡

```
def unsubscribe(self,  
                instrument_tokens)
```

Unsubscribe the given list of
instrument_tokens.

- instrument_tokens is
list of
instrument_tokens to
unsubscribe.

[SHOW SOURCE](#) ≡

Sub-modules

[kiteconnect.exceptions](#)
[\(exceptions.m.html\)](#)

exceptions.py

Exceptions raised by the Kite
Connect client.

:copyright: (c) 2021 by Zerodha
Technology. :license: see
LICENSE for details.

Documentation generated by [pdoc 0.3.2](https://github.com/BurntSushi/pdoc) (<https://github.com/BurntSushi/pdoc>).

pdoc is in the public domain with the [UNLICENSE](http://unlicense.org) (<http://unlicense.org>).

Design by [Kailash Nadh](http://nadh.in) (<http://nadh.in>).